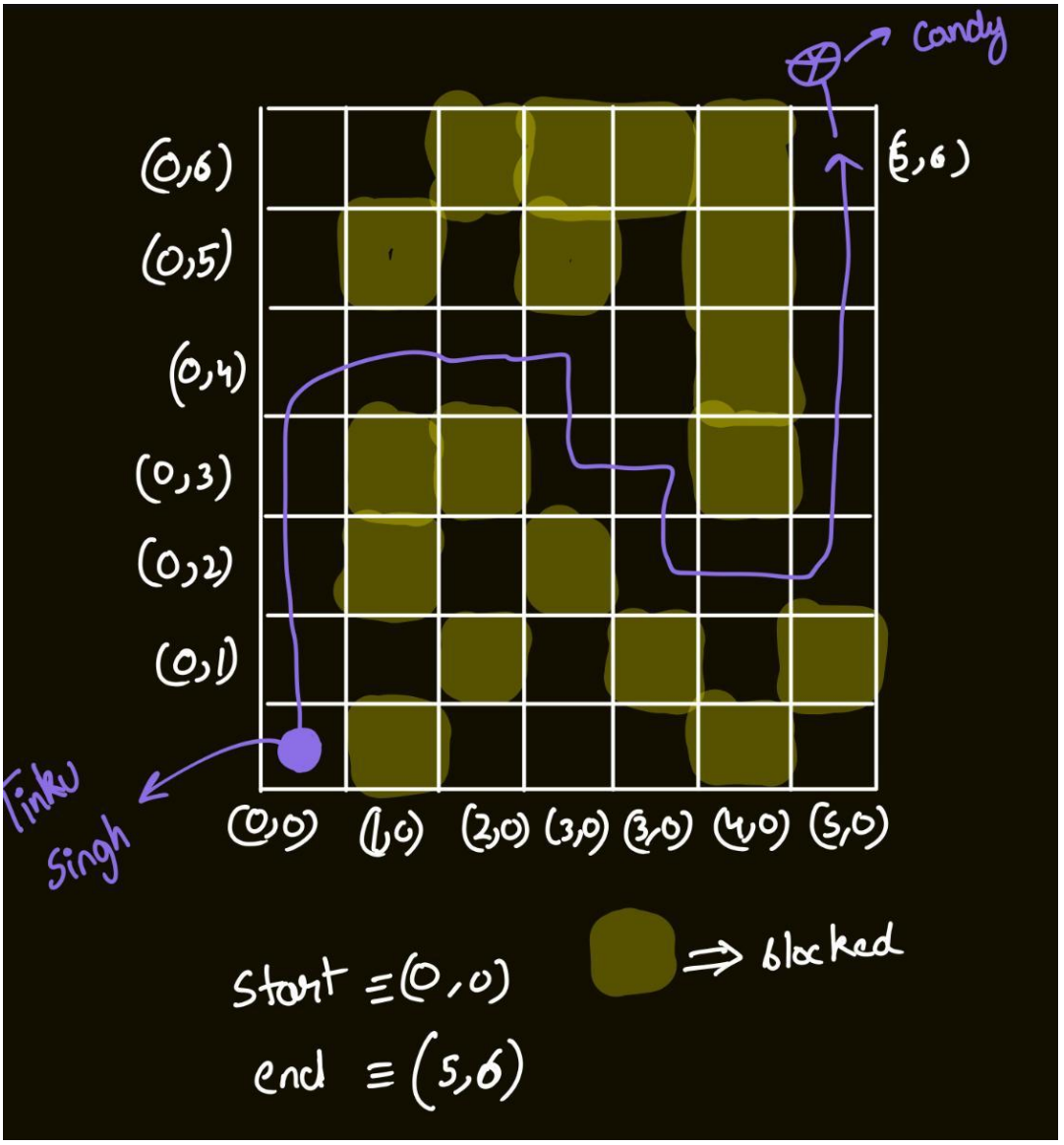
Task 8 — Maze-Solving Algorithms

Comparison

We will use DFS algorithm for this

- DFS explained
    - **Main idea**
        - Go as deep as possible down one path. When you hit a dead end, go back to the last place where you had another choice, and try that instead. Keep doing this until you reach the goal.
    - **What the robot must remember**
        - **Visited cells:** a grid marking which cells it has already been in.
        - **Stack (the path so far):** a list of the cells it passed through, in order. The newest is on top. This is how it knows where to backtrack.
        - **Current position and heading:** which cell it is in, and which way it faces (from the encoders).
        - **Walls seen:** which sides of each cell are open or blocked (from the sensors).
    - **Steps**
        - Start at the start cell. Mark it visited and push it on the stack.
        - Look at the neighbours: which sides are open (no wall) **and** not yet visited?
        - **If there is one or more:** pick one using a fixed order (for example: straight, left, right). Move into it, mark it visited, push it on the stack.
        - **If there is none (dead end, or all neighbours visited):** pop the top cell off the stack and move back to the cell below it. This is **backtracking**.
        - Repeat from step 2 until the current cell is the goal.
        - When the goal is reached, the stack holds the path from start to goal.
    - Understanding with an example …
        - **Rule:** a **node** is any place where the robot must decide something or where a path ends. An **edge** is the stretch of path between two nodes.
        - **Nodes:** the start, the goal, dead ends, and junctions (cells with 3 or more open sides).
        - **Edges:** corridors between nodes. The edge's length is the number of cell steps, or the encoder ticks the robot counts (yaad aaya mene q bola tha ki hame wo n20 motor chaye jisme encoders ho ..?).
        - **Not nodes:** cells in a plain straight corridor or a simple corner. There is only one way to go, so nothing is decided. They are just part of an edge.
        - Consider this maze

- 

- We need to give it a fixed arbitary direction prority say up > right > down > left
- The robot starts at (0,0).
  - It marks (0,0) as visited.
  - It checks its neighbouring cells.
  - Suppose moving up to (0,1) is possible, while moving right is blocked.
  - The robot moves to (0,1) and marks it as visited.
  - At (0,1), suppose the next available cell is (0,2). The robot moves there.
  - It continues moving through available, unvisited cells, such as (0,3) and (0,4).
  - At this stage, the robot is exploring one path without considering whether another path might be shorter.
- Suppose the robot reaches (0,4). From this cell, two neighbouring cells are available:
  - Up: (0,5)
  - Right: (1,4)
  - Since the direction priority is Up, Right, Down, Left, the robot chooses (0,5) first.

        - It does not compare the two paths or calculate which one is better. It simply follows its direction priority and explores the upward path first.
    - Suppose the robot follows the path: (0,4) → (0,5) → (0,6) → (1,6)
        - Now suppose (1,6) has no open, unvisited neighbours.
        - The robot has reached a dead end.
        - DFS handles this situation through a process called backtracking.
        - The robot retraces its path:
        - (1,6) → (0,6) → (0,5) → (0,4)
        - It returns to (0,4), the previous junction.
        - The robot remembers that it has already explored the upward path. Therefore, it does not explore that path again.
        - Instead, it checks for another available neighbour. Since (1,4) is still open and unvisited, the robot moves right to (1,4) and continues exploring.
        - This is the key idea behind DFS: when a path leads to a dead end, the robot goes back to the most recent position where an unexplored path is available.
    - The robot continues repeating the following process:
        - Check the neighbouring cells in the chosen direction order.
        - Move to an open, unvisited neighbour.
        - Mark the new cell as visited and add it to the current path.
        - If no open, unvisited neighbour exists, backtrack.
        - Repeat until the destination (5,6) is reached.
        - The robot does not need to know the entire maze beforehand. It discovers the available paths while exploring.

Exploratory Traversal vs Optimal-Route Replay

- **Exploratory Traversal (Unknown Maze):**
    - The robot does not know the maze in advance. It explores the available paths, detects junctions and dead ends, marks visited cells, and uses DFS with backtracking to discover the maze. It may need to explore multiple paths before reaching the destination.
- **Optimal-Route Replay (Known Map):**
    - Once the maze has been mapped, the robot calculates the shortest route from the start to the destination and follows that route again. It no longer needs to explore alternative paths or perform backtracking.
- **Why is replay simpler?**
    - During replay, the robot already knows which path to take at each junction. Therefore, it only needs to follow the predetermined route and make the required turns, rather than repeatedly exploring and evaluating different paths. This simplifies the decision-making logic and can reduce the time needed to reach the destination.